

Procedimiento de Gestión y Registro de Incidencias

En este documento defino de manera clara cómo he estructurado el control, seguimiento y solución de los fallos técnicos (bugs) surgidos durante el desarrollo de la aplicación web **Chat Home IA**. Llevar esta gestión organizada dentro de Jira es clave para asegurar que el frontend desarrollado en Angular y la API en Flask se comuniquen sin romper funcionalidades previas.

1. Ciclo de Vida del Bug (Flujo de Trabajo en Jira)

Cualquier mal comportamiento detectado en el sistema debe registrarse de forma obligatoria siguiendo un ciclo de vida en nuestro tablero ágil. Los estados por los que pasa una incidencia son los siguientes:

1. **Detección y Apertura (Backlog / To Do):** En el momento en que se identifica un fallo, se abre un ticket en Jira asegurando que el tipo de incidencia sea estrictamente "Bug". Es fundamental rellenar un título descriptivo y detallar los pasos para reproducir el error.
2. **Análisis y Diagnóstico (In Progress):** El desarrollador asignado pasa el ticket a "En curso" cuando empieza a investigar el problema, revisando los logs de los contenedores Docker o las respuestas HTTP en la consola de desarrollo del navegador.
3. **Resolución y Comentario Técnico (Solución Aplicada):** Se modifica el código fuente para corregir el fallo. Antes de dar por solucionado el ticket, se añade un comentario en Jira detallando la corrección técnica exacta (ficheros modificados y lógica aplicada).
4. **Verificación y Cierre (Done):** Tras comprobar localmente que el sistema responde correctamente y no se han generado regresiones en el chat ni en la autenticación, la incidencia se mueve de forma definitiva a la columna de "Finalizada".

2. Tabla de Clasificación de Impacto y Casos Reales

Para priorizar el trabajo de manera lógica, las incidencias se clasifican según su impacto en el funcionamiento global de Chat Home IA. A continuación se muestran los tres niveles definidos junto a los bugs reales solucionados en el proyecto:

Prioridad	Descripción del Impacto	Caso Real Identificado	Solución Técnica Registrada
Alta	Bloquea una funcionalidad crítica del sistema como el acceso o las respuestas principales.	Error 500 al intentar registrar un usuario con un email que ya existía en la base de datos MySQL.	Se implementó un bloque try-catch en el backend de Flask para capturar la excepción de integridad y enviar un código HTTP 409 controlado al cliente.
Alta	El sistema deja de responder debido a fallos o pérdidas de conexión con la infraestructura externa.	Fallo de conexión por timeout prolongado al lanzar consultas pesadas a la API de Groq (Llama 3.1).	Se reajustó el tiempo de espera en el servicio ia_service.py a un límite máximo de 30 segundos y se configuró un mensaje de aviso amigable para el usuario.
Media	Fallo visual o de comportamiento en la interfaz que degrada la experiencia de usuario sin romper la lógica backend.	La ventana del chat de Angular no desplaza el scroll automáticamente hacia abajo al recibir una nueva respuesta de la IA.	Se introdujo el uso de ViewChild en el componente de Angular junto al método scrollToBottom() para forzar el foco inferior del DOM tras actualizar el array de mensajes.

3. Evidencias del Cuadro de Mando (Jira Software)

Para comprobar la trazabilidad completa del desarrollo y asegurar la correcta simulación del flujo ágil, facilito el acceso al proyecto gestionado de forma directa.

Enlace directo al proyecto en Jira (Permiso de Administrador para aperdia310k@g.educaand.es):

<https://fcorcar.atlassian.net/jira/software/projects/P2D/boards/2/timeline?selectedIssue=P2D-45&atlOrigin=eyJpIjoiYmUxMzU0NWRhNDFkNGI4ZGExOTJmMzBjZTE5MWU5MWQlLCJwIjoiajJ9>

A continuación, se adjunta la evidencia visual del bloque de incidencias resueltas dentro del histórico de Sprints, donde se pueden apreciar las etiquetas de prioridad asignadas y los comentarios técnicos de resolución aplicados a cada ticket:

